

ML Project Deployment Report

Football Player Market Value Prediction

Tech Stack

- Python
 - Pandas
 - NumPy
 - Scikit-learn
 - Joblib
 - Streamlit
 - Git & GitHub
-

Work Completed

During this phase, the project was refactored into a modular pipeline with clearly defined functions for each stage of the machine learning workflow. The following functions were implemented:

- `load_data()` – Loads the dataset from the source file and prepares it for further processing.
- `clean_data()` – Handles missing values, removes inconsistencies, and ensures data quality.
- `encode_features()` – Converts categorical variables into numerical format suitable for model training.
- `scale_features()` – Applies feature scaling using `StandardScaler` to normalize numerical inputs.
- `remove_outliers()` – Detects and removes extreme values to improve model stability and performance.
- `feature_engineering()` – Creates new meaningful features from existing data to improve predictive power.
- `feature_selection()` – Selects the most relevant features for training the model.
- `preprocess_pipeline()` – Combines all preprocessing steps into a single reusable pipeline for consistency.
- `train_model()` – Trains the Linear Regression model using the processed dataset and selected features.

Additional work completed:

- Organized the project into a clean and modular folder structure.
 - Converted notebook-based code into reusable Python scripts.
 - Ensured consistent preprocessing between training and prediction stages.
 - Integrated preprocessing steps into a unified pipeline for reliability.
 - Developed a Streamlit application for real-time prediction.
 - Ensured the model works with serialized preprocessing components for deployment.
-

Challenges Faced

One of the main challenges was structuring the preprocessing steps into separate reusable functions while maintaining consistency between training and prediction. Initially, scaling and encoding were applied inconsistently, which affected model performance. This was resolved by introducing a unified `preprocess_pipeline()` function.

Another issue was handling outliers without losing important data, which required careful threshold tuning. Feature selection also required multiple iterations to ensure only relevant attributes were retained for training.

Finally, ensuring that all preprocessing steps worked identically in both training and deployment required restructuring the pipeline to avoid discrepancies.